COMP60272 – Security and Privacy of AI
Coursework II

Zhipeng Wang

Marking. This coursework is marked out of **25 points**.

You will be given a file named `COMP60272_Coursework_II.zip`, which contains the code template. Using the code template, you are required to complete the exercises below and submit all results in a single file named `id_12345678_Coursework_II.zip`.

1 Exercise 1: Federated Learning

Federated Learning (FL) is a distributed optimisation paradigm that enables multiple clients to collaboratively train a shared machine learning model without directly sharing their raw local data. Instead of centralising all training data at a single server, each client performs training locally and only communicates model updates. Formally, consider a model parameterised by $\boldsymbol{w} \in \mathbb{R}^d$. Client i holds a private dataset $\mathcal{D}_i$ and defines a local empirical risk:

$$\mathcal{L}_i(\boldsymbol{w}) = \frac{1}{|\mathcal{D}_i|} \sum_{(x,y)\in\mathcal{D}_i} \ell(f_{\boldsymbol{w}}(x), y)$$

The global optimisation objective is: $\min_{\boldsymbol{w}} \sum_{i=1}^{N} \frac{|\mathcal{D}_i|}{\sum_j |\mathcal{D}_j|} \mathcal{L}_i(\boldsymbol{w})$

1.1 Federated Averaging

One of the most widely used FL optimisation algorithms is Federated Averaging (FedAvg). Training proceeds in communication rounds indexed by $t = 0, 1, \ldots$.

At round t:

1. The server broadcasts the current global model $\boldsymbol{w}^{(t)}$
2. Each client performs E epochs of local Stochastic Gradient Descent (SGD)
3. Clients compute updates $\Delta_i^{(t+1)} = \boldsymbol{w}_i^{(t+1)} - \boldsymbol{w}^{(t)}$ and send them to the server
4. The server aggregates the updates using weighted averaging

   The server computes the aggregated update: $\Delta_{\text{agg}}^{(t+1)} = \sum_{i=1}^{N} \frac{|\mathcal{D}_i|}{\sum_j |\mathcal{D}_j|} \Delta_i^{(t+1)}$

   Then updates the global model: $\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \eta \Delta_{\text{agg}}^{(t+1)}$, where η is the learning rate (typically $\eta = 1$ for FedAvg).

Example. If all clients have equal dataset sizes, then:

$$\Delta_{\text{agg}}^{(t+1)} = \frac{1}{N}\sum_{i=1}^{N}\Delta_i^{(t+1)}, \quad \boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \eta\Delta_{\text{agg}}^{(t+1)}$$

Exercise 1 (6 points). Implement Federated Averaging for MNIST image classification.
Requirements:

- Use at least $N \geq 20$ clients
- Split the MNIST dataset across N clients. The default distribution is IID (Independent and Identically Distributed), where data is randomly shuffled and equally partitioned among clients. You may use other distribution methods if desired.
- Use at least $E \geq 1$ local epochs per round
- Use at least $R \geq 30$ communication rounds
- Implement the standard FedAvg update: $\Delta_i^{(t+1)} = \boldsymbol{w}_i^{(t+1)} - \boldsymbol{w}^{(t)}$ and $\boldsymbol{w}^{(t+1)} = \boldsymbol{w}^{(t)} + \eta\Delta_{\text{agg}}^{(t+1)}$
- Use weighted averaging based on dataset sizes
- The implementation should achieve reasonable test accuracy (typically > 0.90 for MNIST after sufficient training rounds with appropriate hyperparameters)

What to submit.

1. A CSV file:

   `round, accuracy`

   named `solution_1.csv`.

2. The code you wrote to arrive at your solution.

2 Exercise 2: Secure Aggregation

While FL avoids sharing raw data, model updates themselves may still leak sensitive information. Secure aggregation ensures that the server can only observe the *aggregate* of client updates.

Let $\Delta_1, \ldots, \Delta_N$ be the update deltas in a given round. Secure aggregation guarantees the server learns only the weighted aggregate: $\sum_{i=1}^{N} \frac{|\mathcal{D}_i|}{\sum_j |\mathcal{D}_j|}\Delta_i$, without learning individual updates Δ_i.

2.1 Privacy–Robustness Trade-off

Secure aggregation provides strong privacy guarantees but removes the server's ability to inspect individual updates. As a result, malicious behaviour cannot be detected directly.

Example. One large malicious update and many small benign updates are indistinguishable if their sums are equal.

Exercise 2 (4 points). Explain why secure aggregation improves privacy but may complicate robustness. Your report should:

- Clearly explain how secure aggregation improves privacy.
- Clearly explain how secure aggregation may complicate robustness.
- Provide concrete examples or scenarios to illustrate the privacy–robustness trade-off.
- Demonstrate your understanding beyond simply restating the brief (e.g., discuss practical implications, or connections to other concepts).

You may optionally use the code template in `exercise2/` to run demonstrations that support your explanation, but this is not required.

You can also read the following papers to support your report.

- Bonawitz, Keith, et al. "Practical secure aggregation for privacy-preserving machine learning." 2017 ACM SIGSAC Conference on Computer and Communications Security.

  URL: https://eprint.iacr.org/2017/281.pdf

- Bell, James Henry, et al. "Secure single-server aggregation with (poly) logarithmic overhead." 2020 ACM SIGSAC conference on computer and communications security.

  URL: https://eprint.iacr.org/2020/704.pdf

- Lycklama, Hidde, et al. "Rofl: Robustness of secure federated learning." 2023 IEEE Symposium on Security and Privacy (SP).

  URL: https://arxiv.org/pdf/2107.03311

What to submit.

1. A short PDF report (maximum 1 page).

3 Exercise 3: Poisoning Attacks in Secure Federated Learning

Secure aggregation prevents the server from inspecting individual client updates. While this provides strong privacy guarantees, it also enables a powerful class of attacks: *poisoning attacks*, where malicious clients manipulate the training process without being individually observed.

Let $\boldsymbol{w}^{(t)}$ denote the global model at communication round t. Client i computes a local update: $\Delta_i^{(t)} = \boldsymbol{w}_i^{(t+1)} - \boldsymbol{w}^{(t)}$.

A malicious client deviates from the prescribed training procedure and submits a crafted update: $\tilde{\Delta}_i^{(t)} = \mathcal{A}\big(\Delta_i^{(t)}\big)$, where $\mathcal{A}(\cdot)$ denotes an adversarial transformation.

3.1 Threat Model

Assume that:

- A fraction $\rho \in (0, 1)$ of participating clients are malicious
- Malicious clients know the current global model $\boldsymbol{w}^{(t)}$
- The server performs standard Federated Averaging with secure aggregation

Malicious clients do not observe other clients' updates but can coordinate their own behaviour.

3.2 Attack Taxonomy

Poisoning attacks can be categorised along two orthogonal dimensions.

Data Poisoning vs Model Poisoning.

- **Data poisoning:** The attacker modifies local training data but follows the training algorithm honestly.
- **Model poisoning:** The attacker directly crafts arbitrary updates, potentially without performing any training.

Single-shot vs Continuous Attacks.

- **Single-shot attacks:** The attacker injects a malicious update in a single communication round.
- **Continuous attacks:** The attacker participates across multiple rounds and gradually influences the global model.

3.3 Attack Objectives

Poisoning attacks may aim to:

- Degrade the overall test accuracy of the global model
- Induce targeted misclassification for specific inputs
- Implant backdoors that are rarely triggered during benign training

Example: A malicious client may inject a backdoor that causes images containing a specific trigger pattern to be classified as a target label, while preserving high accuracy on normal test data.

Exercise 3 (7 points). Design and implement a poisoning attack against secure federated learning.
The implementation must:

- Use secure aggregation (as in Exercise 2) so the server cannot see individual updates
- Support both benign and malicious clients (specify the fraction ρ of malicious clients)
- Implement at least one attack strategy. You may use the code template to implement your own attack strategy.
- Follow the standard FL update: $\Delta_i^{(t+1)} = \boldsymbol{w}_i^{(t+1)} - \boldsymbol{w}^{(t)}$
- Report appropriate metrics based on the attack type (see CSV format requirements below)

In your implementation, clearly specify:

- Whether the attack is data poisoning or model poisoning
- Whether the attack is single-shot or continuous
- The attack objective (e.g., accuracy degradation, targeted misclassification, or backdoor injection)
- The chosen parameters, including the fraction ρ of malicious clients and why you chose this value (typical values range from 0.1 to 0.3, but you should justify your choice based on attack effectiveness and results)

What to submit.

1. A CSV file named `solution_3.csv` with columns depending on your attack type. The default attack type is **accuracy degradation**, but you are welcome to implement and experiment with other attack types:

- **Accuracy degradation attacks** (default):

  `round, accuracy_clean, accuracy_attack`

  where `accuracy_clean` is the test accuracy without attacks, and `accuracy_attack` is the test accuracy with attacks.

- **Targeted misclassification attacks:**

  `round, accuracy_clean, accuracy_attack, target_class_accuracy`

  where `target_class_accuracy` is the accuracy on the target class (the class you want to misclassify). You may also include additional columns if needed (e.g., `source_class_accuracy` if you're targeting specific source-to-target misclassifications).

- **Backdoor attacks:**

  `round, accuracy_clean, accuracy_attack, backdoor_success_rate`

  where `accuracy_clean` is the accuracy on normal test data, `accuracy_attack` is the accuracy on normal test data with the backdoor model, and `backdoor_success_rate` is the success rate of the backdoor (percentage of triggered samples classified as the target label).

2. The code you wrote to arrive at your solution.

4 Exercise 4: Defenses via Zero-Knowledge Proofs

Secure aggregation hides individual updates. Zero-Knowledge Proofs (ZKPs) can be used to enhance privacy.

4.1 ZKP-based Input Validation

Each client proves that their update delta satisfies a norm bound:

$$\|\Delta_i^{(t+1)}\|_p \leq B$$

where $\Delta_i^{(t+1)} = \boldsymbol{w}_i^{(t+1)} - \boldsymbol{w}^{(t)}$ is the update delta, $\|\cdot\|_p$ denotes the p-norm (typically $p = 2$ for L_2 norm), and B is a predefined bound.

The ZKP allows the server to verify this constraint without learning the actual update $\Delta_i^{(t+1)}$, thus maintaining privacy while filtering out malicious updates that violate the norm bound.

Limitations of ZKP-based Defense. ZKP-based input validation can only filter out attacks that produce updates violating the norm bound $\|\Delta_i^{(t+1)}\|_p \leq B$. However, it cannot defend against all attack types:

- **Accuracy degradation attacks** with large update norms may be filtered out if the bound B is set appropriately, but attackers who know B can adjust their attack strength to satisfy the constraint.
- **Targeted misclassification attacks** may or may not be filtered depending on the attack strength and the chosen bound B.
- **Backdoor attacks** typically produce small-norm updates (to preserve normal accuracy) and may bypass ZKP validation if their updates satisfy $\|\Delta_i^{(t+1)}\|_p \leq B$.

The choice of bound B involves a trade-off: too strict and legitimate updates may be rejected, too loose and attacks may bypass the defense. You should discuss this trade-off and the effectiveness of ZKP against different attack types in your report.

4.2 Evaluation Metrics

To evaluate the overhead introduced by ZKPs, you must report computational and communication costs per communication round.

Each row of the CSV file must contain:

```
round, time_no_zkp, time_zkp, comm_no_zkp, comm_zkp
```

Definition of fields.

- `round`: communication round index
- `time_no_zkp`: total wall-clock time (seconds) per round without ZKPs
- `time_zkp`: total wall-clock time (seconds) per round with ZKPs
- `comm_no_zkp`: total communication volume (bytes) per round without ZKPs
- `comm_zkp`: total communication volume (bytes) per round with ZKPs

Example. For $N = 10$ clients, in round 5:

```
5, 2.31, 3.95, 4800000, 4900000
```

Exercise 4 (8 points). Propose a ZKP-based input validation mechanism for secure FL and evaluate its overhead.
This exercise extends Exercise 3 by adding ZKP-based defense. You should:

- Use the same federated learning setup as Exercises 1–3 (MNIST classification)
- Use at least $N \geq 20$ clients (same as Exercise 1)
- Use at least $R \geq 30$ communication rounds (fewer rounds are acceptable for overhead evaluation)
- Include both benign and malicious clients (specify the fraction ρ of malicious clients, default is 0.2 in the code template)
- Implement ZKP-based input validation that verifies $\|\Delta_i^{(t+1)}\|_p \leq B$ without revealing the update
- Compare the system with and without ZKP to measure overhead
- Test ZKP effectiveness against attacks (e.g., accuracy degradation attacks from Exercise 3)

What to submit.

1. A PDF report (maximum 1 page), which explains your ZKP design, parameter choices, effectiveness analysis, and overhead evaluation

   Your report should:

   - Explain how you design the ZKP mechanism (e.g., norm type, bound B, proof generation and verification)
   - Justify your choice of parameters (e.g., why you chose a particular value for bound B)
   - Discuss the effectiveness of ZKP against different attack types (e.g., accuracy degradation, targeted misclassification, backdoor attacks)
   - Analyze the trade-off in choosing bound B (too strict vs. too loose)
   - Report the computational and communication overhead introduced by ZKPs

2. A CSV file named `solution_4.csv` containing the overhead metrics
3. The code you wrote to arrive at your solution.

Coursework Submission

Your final submission should be packaged in a single ZIP file, named according to your student ID. For example, if your student ID is 12345678, you should name your file `id_12345678_Coursework_II.zip`. The content of the ZIP file should be structured as follows:

File naming conventions:

- `solution_1.csv`: CSV file for Exercise 1
- `exercise1/`: Directory containing your code for Exercise 1
- `report_2.pdf`: PDF report for Exercise 2 (maximum 1 page)
- `solution_3.csv`: CSV file for Exercise 3
- `exercise3/`: Directory containing your code for Exercise 3
- `report_4.pdf`: PDF report for Exercise 4 (maximum 1 page)
- `solution_4.csv`: CSV file for Exercise 4
- `exercise4/`: Directory containing your code for Exercise 4

Submissions with incorrect filenames or structure will be penalised.